

Lukasz

Osoba 1 - Matematyka 3D, graf sceny, budowa pokoju

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/Math3D.h, Engine/Math3D.cpp, Engine/SceneNode.h, Engine/SceneNode.cpp, Demo/Room.cpp

0. Gdzie sa moje pliki (mapa do prezentacji)

Ponizsze drzewo pokazuje caly projekt. Strzałka <<< wskazuje pliki, ktore robilem ja (Lukasz). W trakcie pokazu otwieraj wlasnie te.

```
pgk/
|
+-- Engine/                                <- silnik (rdzen)
|   +-- Math3D.h                          <<< MOJE  (wektory, macierze, raycasty)
|   +-- Math3D.cpp                        <<< MOJE
|   +-- SceneNode.h                      <<< MOJE  (graf sceny - drzewo obiektow)
|   +-- SceneNode.cpp                   <<< MOJE
|   +-- Camera.* Light.* Texture.*      (Kacper)
|   +-- PrimitiveNode.*                 (Rogert)
|   +-- Engine.*                        (Jakub)
|
+-- Demo/                                <- gra
|   +-- Room.cpp                        <<< MOJE  (sciany + dekoracje pokoju)
|   +-- Obstacles.cpp                  (Rogert)
|   +-- Targets.cpp                    (Kacper)
|   +-- Gameplay.cpp Constants.h ShootingGallery.h (Jakub)
|
+-- main.cpp CMakeLists.txt README.md  (Jakub)
```

Najwazniejsze do zapamietania: ja odpowiadam za **matematyke** (wszystko co liczy wektory i macierze), za **strukture sceny** (jak obiekty sa ulozone w drzewo) i za **zbudowanie pomieszczenia** w grze.

1. O co chodzi w moich plikach (po ludzku)

Silnik 3D musi cos policzyc, zanim cokolwiek narysuje. Zeby pokazac szescian w odpowiednim miejscu i pod odpowiednim katem, trzeba mnozyc **macierze** i przekształcac **wektory**. To wlasnie robi **Math3D** - to nasz wlasny, reczny zestaw narzedzi matematycznych (nie uzywamy gotowej biblioteki jak glm).

SceneNode to z kolei sposob, w jaki trzymamy obiekty sceny. Zamiast luznej listy, obiekty tworza **drzewo** (graf sceny). Dzieki temu jak ruszymy rodzicem, dzieci ruszaja sie razem z nim.

Room.cpp to juz czesc gry - uzywam swoich klas (i prymitywow kolegow), zeby zbudowac pokoj: podloge, sufit, 4 sciany i dekoracje.

2. Math3D.h / Math3D.cpp - matematyka silnika

2.1. Vec3 - wektor 3D

Wektor to po prostu trzy liczby: x, y, z. Moze oznaczac **punkt** w przestrzeni albo **kierunek**. Zdefiniowalem podstawowe operacje (dodawanie, odejmowanie, mnozenie przez liczbe) plus funkcje matematyczne.

Math3D.cpp - kluczowe operacje na wektorach

```
float dot(const Vec3& a, const Vec3& b) {
    return a.x*b.x + a.y*b.y + a.z*b.z;    // iloczyn skalarny
}

Vec3 cross(const Vec3& a, const Vec3& b) { // iloczyn wektorowy
    return Vec3(a.y*b.z - a.z*b.y,
                a.z*b.x - a.x*b.z,
                a.x*b.y - a.y*b.x);
}

float length(const Vec3& v) { return std::sqrt(dot(v, v)); }

Vec3 normalize(const Vec3& v) {
    float len = length(v);
    if (len <= 0.000001f) return Vec3(0,0,0); // ochrona przed dzieleniem przez 0
    return v / len;
}
```

dot (iloczyn skalarny) mowi nam m.in. o kacie miedzy wektorami. **cross** (iloczyn wektorowy) daje wektor **prostopadly** do obu - uzywam go np. do policzenia osi kamery. **normalize** skraca wektor do dlugosci 1, zachowujac kierunek (wektory kierunku prawie zawsze chcemy znormalizowane).

2.2. Mat4 - macierz 4x4 (najwazniejsza rzecz!)

Macierz 4x4 to 16 liczb, ktore opisuja przekształcenie: przesuniecie, obrot, skalowanie albo rzutowanie. Trzymam je w tablicy **column-major** (kolumnami), bo tego wymaga OpenGL.

Math3D.cpp - dostep do elementu macierzy

```
float& Mat4::at(int row, int col) { return m[col * 4 + row]; }
//                                     ^^^^^^^^^^^^^^^^^
//   element (row,col) jest pod indeksem col*4+row => column-major
```

Dlaczego column-major? Bo funkcja OpenGL *glLoadMatrixf* czyta dane wlasnie w tej kolejnosci. Gdybym trzymal je wierszami (row-major), obraz bylby kompletnie przekrecony. To wazne - wykladowca moze o to spytac.

Macierz lookAt (i slynny bug, ktory naprawilismy)

lookAt tworzy macierz widoku - ustawia 'oko' kamery w punkcie *eye*, kierując je na *center*. Liczymy trzy osie kamery: *forward* (do przodu), *side* (w prawo) i *trueUp* (w gore).

Math3D.cpp - lookAt (wersja poprawna)

```
Vec3 forward = normalize(center - eye);
Vec3 side    = normalize(cross(forward, up));
Vec3 trueUp  = cross(side, forward);

Mat4 result = Mat4::identity();
result.at(0,0)=side.x; result.at(0,1)=side.y; result.at(0,2)=side.z; // wiersz 0
result.at(1,0)=trueUp.x;result.at(1,1)=trueUp.y;result.at(1,2)=trueUp.z; // wiersz 1
result.at(2,0)=-forward.x; result.at(2,1)=-forward.y; result.at(2,2)=-forward.z;
result.at(0,3)=-dot(side,eye);
result.at(1,3)=-dot(trueUp,eye);
result.at(2,3)= dot(forward,eye);
```

Bug, który mieliśmy: wektory osi były wpisane jako **kolumny** zamiast **wierszy**. Dla obrotu transpozycja = odwrotność, więc efekt był taki, że kamera 'krazyła wokół jakiegoś dziwnego punktu', a strzały nie trafiały w cele (bo kierunek patrzenia nie zgadzał się z tym, co widac). Przy yaw=pitch=0 bug był niewidoczny, bo zamieniane pola były zerami - dlatego długo go nie zauważyliśmy.

2.3. Funkcje przecięć promienia (raycasty)

To one sprawiają, że w grze można w coś 'trafic'. Promień to po prostu: $P(t) = \text{origin} + t * \text{kierunek}$. Szukamy najmniejszego dodatniego t , przy którym promień dotyka obiektu.

Math3D.cpp - przecięcie promienia ze sferą

```
bool raySphereIntersect(const Vec3& origin, const Vec3& dir,
                        const Vec3& center, float radius, float& outT) {
    Vec3 oc = origin - center;
    float a = dot(dir, dir);
    float b = dot(oc, dir);
    float c = dot(oc, oc) - radius*radius;
    float disc = b*b - a*c;           // wyznik równania kwadratowego
    if (disc < 0.0f) return false;    // brak przecięcia
    float sqrtD = std::sqrt(disc);
    float t1 = (-b - sqrtD) / a;      // bliższe
    float t2 = (-b + sqrtD) / a;      // dalsze
    if (t1 > 0.0001f) { outT = t1; return true; }
    if (t2 > 0.0001f) { outT = t2; return true; }
    return false;
}
```

Do zwykle **równanie kwadratowe**: podstawiamy $P(t)$ do równania sfery $|P - \text{center}|^2 = r^2$ i wychodzi $a*t^2 + 2*b*t + c = 0$. Jeżeli wyznik (disc) jest ujemny - promień mija sferę. Mam też analogiczne funkcje dla walca (*rayCylinderIntersect*), stożka (*rayConeIntersect*) i prostopadłościanu (*rayAABBIntersect* - metoda 'slab'). Kacper i Rogert używają ich w grze.

3. SceneNode.h / SceneNode.cpp - graf sceny

Graf sceny to **drzewo** obiektów. Każdy obiekt (SceneNode) ma rodzica i listę dzieci. Dzięki temu transformacje się **dziedziczą**: jak obroci rodzica, to dzieci obraca się razem z nim. To standard w silnikach 3D.

3.1. Macierz lokalna

SceneNode.cpp - sklejenie pozycji, obrotu i skali w jedną macierz

```
Mat4 SceneNode::localMatrix() const {
    return Mat4::translation(localPosition)
        * Mat4::rotationY(localRotation.y)
        * Mat4::rotationX(localRotation.x)
        * Mat4::rotationZ(localRotation.z)
        * Mat4::scale(localScale);
}
```

Kolejność mnożenia ma znaczenie! Czytając od prawej: najpierw **skalujemy** obiekt, potem **obracamy** (Z, X, Y), na końcu **przesuwamy**. Gdyby przesunięcie było pierwsze, obiekt obracałby się wokół środka świata, a nie wokół siebie.

3.2. Rekurencyjne rysowanie

SceneNode.cpp - przejście po całym drzewie sceny

```
void SceneNode::renderRecursive(const Mat4& parentMatrix) const {
    Mat4 world = parentMatrix * localMatrix(); // moja macierz świata
    if (nodeVisible) renderSelf(world);        // narysuj siebie (jeżeli widoczny)
    for (const auto& child : childNodes)      // potem wszystkie dzieci
        child->renderRecursive(world);
}
```

Silnik wola te funkcje na korzeniu (root) z macierzą jednostkową. Funkcja schodzi w dół drzewa, mnożąc macierze. **renderSelf** jest puste w klasie bazowej - dopiero klasy potomne (szescian, sfera, światło) nadpisują je i rysują geometrie. To przykład **polimorfizmu** i wzorca, gdzie klasa bazowa zarządza strukturą, a potomne definiują szczegóły.

nodeVisible - jak ustawimy na false, obiekt się nie rysuje (ale jego dzieci tak). Kacper używa tego do 'puli celów' - chowamy sfery zamiast je usuwać.

4. Demo/Room.cpp - budowa pomieszczenia

Tu używam wszystkiego powyżej plus prymitywów Rogerta (PlaneNode, SphereNode...). Buduje pokój o wymiarach 20 x 6 x 30 metrów: podłogę, sufit i 4 ściany, każda jako **PlaneNode** obrocony w odpowiednią stronę.

Room.cpp - przykład jednej ściany (sufit)

```
// sufit: płaszczyzna obrócona o 180 stopni (PI) wokół X, żeby normalna
// patrzyła w dół (do wnętrza pokoju)
ceiling_>setPosition(Vec3(0, ROOM_HEIGHT, ROOM_Z_CENTER));
ceiling_>setRotation(Vec3(PI, 0, 0));
ceiling_>setUVScale(FLOOR_UV_SCALE);
ceiling_>setMaterial(ceilingMat);
ceiling_>setTexture(ceilingTex_);
```

buildDecorations() dorzuca dekoracje: kule i kostki na ścianach, torus (zyrandol) pod sufitem. Używam lambda (makeSphere, makeCube, makeCylinder), żeby nie powtarzać kodu. Dekoracje są wysoko (y >= 3), więc gracz pod nimi przechodzi i nie ma z nimi kolizji.

Tekstury sa albo wczytane z pliku BMP (jezeli istnieje), albo wygenerowane proceduralnie - np. *generatePerlinNoise* dla sufitu. Te funkcje napisal Kacper, ja ich tylko uzywam.

5. Pytania od wykładowcy - przygotowanie

Math3D

P: Dlaczego trzymasz macierz jako tablice 16 floatow, a nie jako tablica 4x4?

O: Bo OpenGL (`glLoadMatrixf` / `glMultMatrixf`) przyjmuje płaską tablicę 16 floatów. Trzymanie ich od razu w takiej formie pozwala podać wskaźnik `data()` wprost do OpenGL bez konwersji. Dostęp (`row,col`) realizuje funkcja `at` jako `m[col*4+row]`.

P: Co to znaczy column-major i dlaczego to ważne?

O: Column-major znaczy, że kolejne 4 liczby w pamięci to jedna kolumna macierzy. OpenGL tego wymaga. Gdyby trzymać dane wierszami (row-major), macierz byłaby transponowana i wszystkie transformacje byłyby złe - obraz przekrecony.

P: Wyjaśnij działanie raySphereIntersect linijką po linijce.

O: Podstawiam promień $P(t) = \text{origin} + t * \text{dir}$ do równania sfery $|P - \text{center}|^2 = r^2$. Po rozwinięciu wychodzi równanie kwadratowe $a * t^2 + 2b * t + c = 0$, gdzie $a = \text{dot}(\text{dir}, \text{dir})$, $b = \text{dot}(\text{oc}, \text{dir})$, $c = \text{dot}(\text{oc}, \text{oc}) - r^2$ ($\text{oc} = \text{origin} - \text{center}$). Liczę wyznik $\text{disc} = b^2 - a * c$. Jeżeli < 0 - brak przecięcia. Inaczej liczę dwa pierwiastki t_1, t_2 i wybieram najmniejszy dodatni (najbliższy punkt przed kamerą).

P: Dlaczego sprawdzasz $t > 0.0001f$, a nie $t > 0$?

O: Mały epsilon chroni przed błędami zaokrągleń float i przed 'trafieniem' w punkt startowy promienia ($t=0$). Bez tego mogłyby pojawiać się fałszywe przecięcia tuż przy kamerze.

P: Co robi cross (iloczyn wektorowy) i gdzie go używasz?

O: cross daje wektor prostopadły do dwóch podanych. Używam go w lookAt: mając kierunek patrzenia (forward) i przybliżoną górę (up), liczę prawo kamery $\text{side} = \text{cross}(\text{forward}, \text{up})$, a potem dokładną górę $\text{trueUp} = \text{cross}(\text{side}, \text{forward})$. Tak buduje 3 prostopadłe osie kamery.

P: Dlaczego w normalizacji sprawdzasz długość przed dzieleniem?

O: Żeby nie dzielić przez zero. Wektor zerowy ma długość 0 - dzielenie dałoby nieskończoność/NaN. Zwracam wtedy wektor zerowy jako bezpieczną wartość.

SceneNode

P: Po co graf sceny? Nie wystarczy lista obiektów?

O: Graf pozwala na hierarchie - obiekt-dziecko dziedziczy transformację rodzica. Np. światło doczepione do obiektu rusza się razem z nim. Mnożenie macierzy $\text{rodzic} * \text{dziecko}$ robi to automatycznie. Z płaskiej listy trzeba by ręcznie pamiętać zależności.

P: Dlaczego renderRecursive mnoży parentMatrix * localMatrix, a nie odwrotnie?

O: Bo macierz świata dziecka = macierz świata rodzica złożona z lokalną transformacją dziecka. Kolejność $\text{rodzic} * \text{dziecko}$ sprawia, że lokalna transformacja działa 'wewnątrz' układu rodzica.

P: Czemu localMatrix mnozy w kolejnosci $T * R * S$?

O: Czytajac od prawej (bo tak dzialaja macierze): najpierw skala, potem obrot, na koncu translacja. Dzieki temu obiekt skaluje i obraca sie wokol wlasnego srodka, a dopiero potem jest przesuwany na miejsce. Odwrotna kolejnosc daloby obrot wokol srodka swiata.

P: Czym jest renderSelf i dlaczego jest wirtualne?

O: To metoda, ktora rysuje konkretna geometrie. W SceneNode jest pusta (wezel grupujacy). Klasy potomne (CubeNode, PointLight...) nadpisuja ja. Wirtualnosc pozwala wywolac wlasciwa wersje przez wskaznik do klasy bazowej - to polimorfizm.

P: Co daje flaga nodeVisible?

O: Gdy false, renderSelf jest pomijane (obiekt znika), ale dzieci nadal sie rysuja. Uzywamy tego do puli celow - zamiast tworzyć i niszczyć sfery, po prostu je chowamy/pokazujemy.

Room.cpp

P: Dlaczego sufit jest obrocony o PI (180 stopni)?

O: PlaneNode ma normalna skierowana w gore (+Y). Sufit musi swiecic do wnetrza pokoju, czyli w dol. Obrot o 180 stopni wokol osi X odwraca normalna, zeby oswietlenie i sciany dzialaly poprawnie od spodu.

P: Co to jest setUVScale i po co?

O: Skaluje wspolrzedne tekstury. Domyslnie 1 powtorzenie tekstury na 1 metr - na scianie 30m daloby 30 malych powtorzen (brzydko). Ustawiam mniejsza wartosc (np. 0.2), zeby kafelki byly wieksze. To metoda z PlaneNode (Rogert), ja ja tylko stosuje.

P: Czemu dekoracje nie maja kolizji?

O: Sa umieszczone wysoko ($y \geq 3m$), a gracz ma oczy na 1.75m i hitbox tylko w poziomie. Wiec fizycznie nie da sie w nie wejsc - gracz po prostu pod nimi przechodzi. Nie trzeba dla nich liczyć kolizji, co oszczedza kod.